

AIR — Agentic Incident Response

MVP Engineering Specification — Feature Architecture, Data Model, APIs & AI Design

Document type	Engineering specification (feature-based)
Audience	Development team, solution architects, AI/ML engineers
Status	Ready for architecture review
Scope	Features 1–6 (MVP): Alert Intake, Investigation Planner, Evidence Collection, Root Cause Assessment, Resolution Planning, AI Incident Report
Source	Consolidated from architecture design notes; restructured for engineering review and implementation

How to read this document

This specification is organized as one page per feature, matching how it should be split into child pages in Confluence. Each feature chapter follows the same structure: Objective & Scope, Architecture, Data Model, API Specification, AI/Prompt Design (where applicable), Design Decisions, and Definition of Done.

Blue callouts mark **MVP scope** (build now). Gray callouts mark **Enterprise / post-MVP** scope (design for, do not build yet).

Table of Contents

- 1. Product Overview & System Architecture
- 2. Feature 1 — Alert Intake & Incident Initialization

- 3. Feature 2 — Investigation Planner Agent
- 4. Feature 3 — Evidence Collection Framework
- 5. Feature 4 — Root Cause Assessment
- 6. Feature 5 — Resolution Planning Agent
- 7. Feature 6 — AI Incident Report & Knowledge Base
- 8. Appendix — Cross-Cutting Design Decisions & Roadmap

Recommended Confluence structure: create this as a parent page “AIR — Agentic Incident Response” containing Section 1, with Sections 2–7 each imported as a child page (Feature 1 ... Feature 6, then Appendix). Anchors in this document map 1:1 to those child pages.

1. Product Overview & System Architecture

1.1 Executive Summary

AIR (Agentic Incident Response) is an autonomous AI SRE platform that investigates production incidents from alerts, gathers evidence across observability and engineering systems, formulates a root cause hypothesis, validates it against evidence, recommends remediation, and produces a structured post-incident report.

AIR is not a single LLM agent. It is a multi-agent investigation platform that orchestrates multiple specialized agents, each with one responsibility, coordinated through a LangGraph workflow. Unlike rule-based AIOps tools, AIR reasons over production context using agentic workflows, deterministic evidence correlation, and LLM-guided investigation, with every AI conclusion traceable back to the evidence that produced it.

1.2 Product Scope

In scope
Alert investigation, root cause analysis, evidence collection, historical search, runbook retrieval, Slack integration, ServiceNow integration, Jira integration, LLM-guided investigation, recommendation generation, human approval.

Product Roadmap

Phase	Contents
Phase 1 (MVP)	Alert intake, planner, evidence collection, RCA generation, Slack updates, ServiceNow integration, human approval. This document specifies Phase 1 in full.
Phase 2	Multi-hypothesis investigation, causal dependency graph, historical incident learning, evaluation framework, richer observability integrations.
Phase 3	Autonomous remediation with approval policies, deployment monitoring, proactive anomaly detection, continuous background agents, production knowledge graph.

1.3 High-Level Investigation Flow

Rather than prompting a single LLM to "find the root cause," AIR decomposes the investigation into a pipeline of specialized steps, each owned by a distinct agent or deterministic service:

Alert Arrives → Planner Agent → Parallel Evidence Collection → Data Normalization

→ Correlation Agent → Hypothesis Generation → Specialized Investigation Agents

→ Evidence Scoring → Refinement Loop → Historical Knowledge → Causal Timeline

→ Root Cause Ranking → Recommendation Agent

In the LangGraph implementation, each stage becomes a graph node:

```

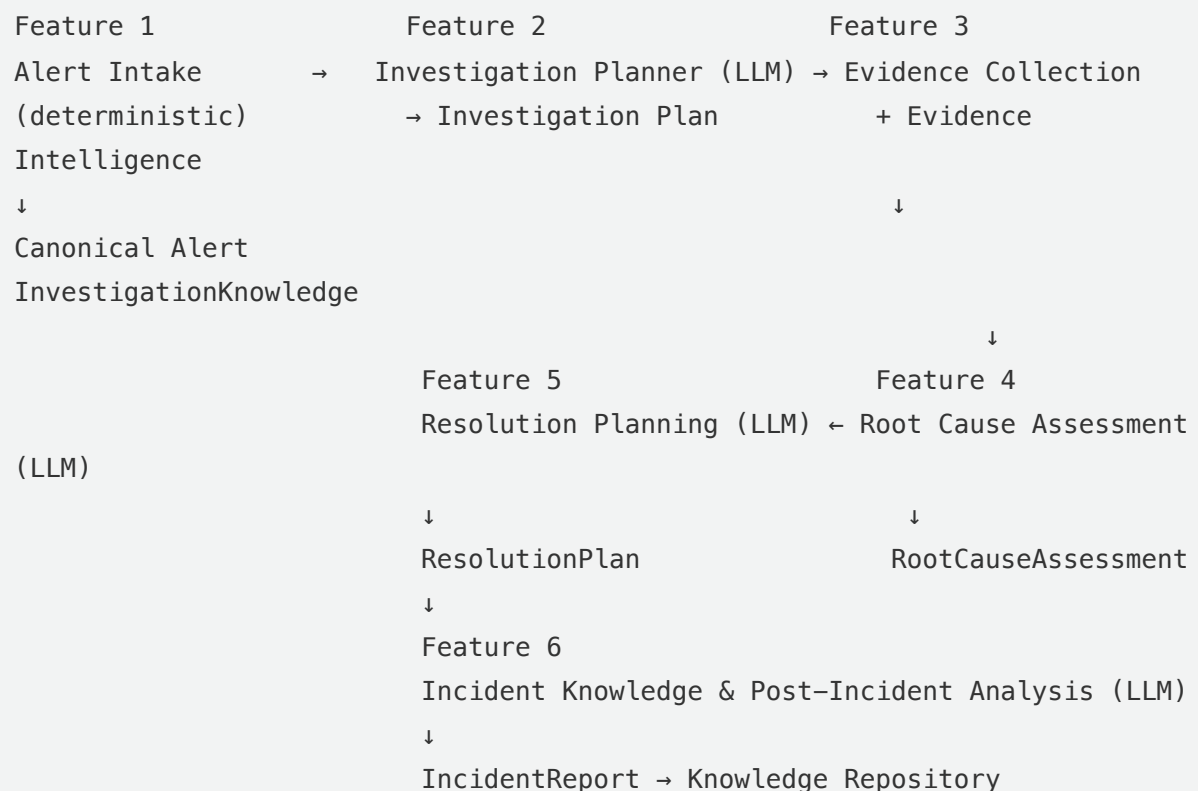
ingest_incident, triage_incident, plan_investigation, collect_logs,
collect_metrics,
collect_traces, collect_deployment, collect_k8s_events,
retrieve_similar_incidents,
retrieve_runbooks, correlate_evidence, generate_hypotheses,
investigate_hypothesis
(parallel subgraph), validate_evidence, build_causal_timeline,
rank_root_causes,
generate_recommendations, human_approval, execute_remediation, verify_fix,
close_incident

```

MVP SCOPE

The MVP implements a linear subset of this flow across six features: Alert Intake → Investigation Planner → Evidence Collection → Root Cause Assessment → Resolution Planning → AI Incident Report. Multi-hypothesis competition, the causal dependency graph, autonomous remediation, and continuous learning are Phase 2/3 and are called out as Enterprise scope throughout this document.

1.4 MVP End-to-End Architecture



Three architectural principles recur across every feature and should be treated as platform-wide standards, not per-feature choices:

Principle	Rule
Evidence → Knowledge → Reasoning	Facts collected from tools (Evidence) are deterministically consolidated into relationships and context (Knowledge) before any LLM reasons over them (Reasoning). The LLM never sees raw telemetry.
Context Builder / Prompt Builder / LLM Service	Every AI-enabled feature follows the same layering: a Context Builder assembles business data, a Prompt Builder renders it into a versioned prompt template, and a single shared LLM Service executes the call, handles retries, and validates structured output. No agent or controller calls the OpenAI SDK directly.
Thin LangGraph nodes	LangGraph nodes orchestrate; services implement. A node loads state, calls a service, and writes state back — it never embeds prompt text, SQL, or business rules.

1.5 Technology Stack

The following stack applies across all six features and should be treated as the platform's standing technical baseline.

Frontend

Layer	Technology	Purpose
UI Application	React 19 + TypeScript	Component-based web application
UI Framework	Material UI (MUI)	Enterprise UI components, accessibility
State Management	TanStack Query + Zustand	Server cache + lightweight client state
Workflow Visualization	React Flow	Visualize the investigation/LangGraph execution
Charts	Recharts	Incident metrics and analytics
Code/JSON Viewer	Monaco Editor	Logs, JSON, YAML, prompt inspection

Backend API Layer

Layer	Technology	Purpose
API Gateway	Spring Boot 3.5 (Java 21)	Central public API gateway
Authentication	Spring Security + OAuth2 + JWT	Enterprise SSO and RBAC
Validation	Jakarta Validation	Request validation
API Documentation	OpenAPI 3 + Swagger	Interactive API docs
Real-time Communication	REST + WebSocket + SSE	Live investigation updates

AI Platform

Layer	Technology	Purpose
Language	Python 3.12+	AI service implementation
Orchestration	LangGraph	Multi-agent workflows, checkpoints, retries, human approval
Tool Framework	LangChain	Tool integrations only
API Framework	FastAPI	AI service internal APIs
LLM SDK	OpenAI SDK (Responses API)	Model integration
Data Validation	Pydantic v2	Structured data models
Structured Output	OpenAI Structured Outputs	Guaranteed JSON schema conformance
Prompt Management	LangSmith / Prompt Repository	Prompt version control

Large Language Models

Use Case	Model
Investigation planning, RCA, recommendation generation, report generation	GPT-5.5
Lightweight tasks (cost optimization)	GPT-4.1 (optional)

Data, Messaging & Infrastructure

Layer	Technology	Purpose
Primary Database	PostgreSQL 16	Operational system of record
Vector Database	pgvector	Semantic search (Enterprise)
Cache	Redis	Session, prompt, state cache
Blob Storage	AWS S3 / Azure Blob	Incident attachments
Event Streaming	Apache Kafka	Incident event bus, async processing, DLQ
Schema Registry	Confluent Schema Registry	Event versioning
Containerization	Docker	Application packaging
Orchestration	Kubernetes / OpenShift	Container orchestration
IaC	Terraform + Helm	Infrastructure provisioning & deployment
CI/CD	GitHub Actions / Jenkins	Automated build & deploy

Observability & Security

Layer	Technology	Purpose
Metrics / Dashboards	Prometheus + Grafana	Platform monitoring & visualization
Tracing	OpenTelemetry + Jaeger	Distributed trace of agent execution
Logging	Loki / Elasticsearch	Centralized logs
AI Metrics	LangSmith	Prompt, token, and workflow analytics
AuthN/AuthZ	OAuth2 + OIDC + RBAC	User authentication & access control
Secrets Management	HashiCorp Vault / Kubernetes Secrets	Secure secret storage — never hardcode credentials
Encryption	TLS 1.3	Secure communication

External Integrations

All observability and engineering-system integrations (Prometheus, Grafana, Elastic/Splunk, Datadog, Kubernetes, GitHub, Slack, ServiceNow, Jira) shall be implemented behind the Model Context Protocol (MCP) or direct REST adapters as specified in Feature 3.5, never called directly from an agent.

Recommended Repository Structure

Repository	Stack	Responsibility
air-ui-app	React + MUI	User interface
air-api-app	Spring Boot	API gateway & business services
air-agent-app	Python + LangGraph	AI agents & orchestration
air-shared-sdk	Java/Python	Shared DTOs and utilities
air-mcp-servers	Python	MCP tool integrations
air-infra	Terraform + Helm	Infrastructure automation
air-observability	Grafana + Prometheus	Monitoring and dashboards
air-docs	MkDocs / Docusaurus	Product documentation

AIR-F001

Deterministic · No LLM

2. Feature 1 — Alert Intake & Incident Initialization

2.1 Objective & Scope

Alert Intake is the entry point into the AIR platform. The system shall receive alerts from monitoring tools, validate and normalize each alert, enrich it with enterprise metadata, determine whether an incident already exists, create or update the incident, and trigger the AI investigation workflow. This feature shall not use an LLM: processing is fully deterministic to guarantee low latency, consistency, and horizontal scalability.

In scope
Receive alerts from monitoring tools · validate incoming requests · authenticate monitoring systems · normalize vendor-specific alert formats into a Canonical Alert Model · enrich alerts with enterprise metadata · detect duplicate alerts/incidents · create or update incidents · persist alert and incident records · publish investigation events · initialize the LangGraph investigation.

2.2 High-Level Flow

Monitoring Tool → API Gateway → Alert Intake Service
→ Validate Request → Authenticate Source → Normalize Alert → Enrich Context
→ Detect Duplicate → Persist Alert → Create/Update Incident → Publish Kafka Event
→ Return HTTP 202 → LangGraph Investigation (Feature 2)

2.3 Canonical Alert Model

Every monitoring tool (Prometheus, Datadog, Grafana, CloudWatch, etc.) emits a different payload shape. Feature 1 shall convert every incoming alert into one common model before it reaches any other feature. This model is immutable once created; later features append new artifacts (Investigation Plan, Evidence, Root Cause, Recommendation) around it rather than modifying it.

<pre>{ "schemaVersion": "1.0", "metadata": { "alertId": "ALT-20260805-000001", "externalAlertId": "PROM-981234", "correlationId": "CORR-5c1f8f91-7eaf-4f08-bd8d-c3d2b9b0b123",</pre>
--

```
    "receivedAt": "2026-08-05T10:15:42Z"
  },
  "source": {
    "monitoringTool": "Prometheus",
    "alertRule": "PaymentFailureRateHigh",
    "alertType": "APPLICATION"
  },
  "resource": {
    "serviceName": "payment-service",
    "applicationName": "Digital Payments",
    "environment": "PROD",
    "namespace": "payments-prod",
    "cluster": "aks-prod-eastus",
    "region": "East US",
    "host": "payment-api-7f9f9d9f8d-kx7q2",
    "pod": "payment-api-7f9f9d9f8d-kx7q2"
  },
  "classification": {
    "severity": "CRITICAL",
    "priority": "P1",
    "category": "Application",
    "businessCriticality": "Tier-1"
  },
  "metric": {
    "name": "payment_failure_rate",
    "unit": "%",
    "currentValue": 23.7,
    "threshold": 5.0,
    "comparison": ">",
    "evaluationWindow": "5m"
  },
  "businessContext": {
    "businessProcess": "Customer Payment Processing",
    "customerImpact": "Payment transactions are failing",
    "affectedChannel": "Mobile App, Web Portal",
    "estimatedAffectedUsers": 1850
  },
  "time": {
    "alertStartedAt": "2026-08-05T10:10:00Z",
    "alertReceivedAt": "2026-08-05T10:15:42Z"
  },
  "labels": {
    "team": "Payments",
    "owner": "Payment Platform",
    "technology": "Spring Boot",
    "runtime": "OpenShift"
  },
  "annotations": {
    "summary": "Payment failure rate exceeded threshold.",
```

```

    "description": "Payment failure rate is 23.7% over the last five minutes.
Threshold is 5%.",
    "runbook": "https://wiki.company.com/runbooks/payment-failure"
  },
  "rawPayloadReference": {
    "payloadStored": true,
    "payloadLocation": "alert.raw_payload"
  }
}

```

Section	Purpose	Used by
metadata	Identity and traceability	All features
resource	Where the problem occurred	Planner, Evidence
classification / metric	Severity, priority, triggering metric	Planner
businessContext	Business impact	RCA, Recommendation
rawPayloadReference	Link to original payload	Debugging & audit

The Planner (Feature 2) shall receive a reduced projection of this model, not the full Prometheus payload:

```

{ "serviceName": "payment-service", "environment": "PROD", "severity":
"CRITICAL", "alertType": "APPLICATION",
  "metric": { "name": "payment_failure_rate", "currentValue": 23.7,
"threshold": 5.0 },
  "businessContext": { "businessProcess": "Customer Payment Processing" } }

```

The Canonical Alert Model shall NOT include:

the Investigation Plan (selectedAgents), raw Evidence (logs), Hypotheses (possibleRootCause), or the RCA (rootCause). Those artifacts belong to Features 2–5 and shall be layered around — never merged into — the canonical alert.

2.4 Duplicate Detection Specification

Duplicate Detection is the business decision point of Feature 1: it determines whether an incoming alert creates a new incident or is associated with an existing active incident, preventing incident storms where dozens of alerts for the same production problem generate dozens of incidents. This component shall be 100% deterministic and shall not call an LLM.

Matching strategy

Rather than comparing the complete alert, the system shall compare a small set of stable business keys. Fluctuating values such as CPU % or latency are intentionally excluded from matching.

Field	Required	Why
Service Name	Required	Same application/service
Environment	Required	PROD and UAT shall never correlate
Alert Type	Required	Application vs. Database vs. Infrastructure
Severity	Optional	Configurable per business rules
Business Process	Optional	Useful for large domains

The system shall build a deterministic fingerprint from these stable attributes, e.g. `payment-service|PROD|APPLICATION`, hashed with SHA-256, and store it on both `alert.fingerprint` and `incident.fingerprint`.

Only incidents in an active status (`OPEN`, `INVESTIGATING`, `RCA_COMPLETED`, `REMEDIATION_PENDING`) within a configurable duplicate window (default 15 minutes) shall be considered; `RESOLVED` and `CLOSED` incidents are excluded.

```
SELECT incident_id FROM incident
WHERE fingerprint = :fingerprint
  AND status IN ('OPEN','INVESTIGATING','RCA_COMPLETED','REMEDIATION_PENDING')
  AND opened_at >= NOW() - INTERVAL '15 minutes'
LIMIT 1;
```

Scenario	Expected result
Same service/environment/alert type, incident still OPEN	Link to existing incident
Same service, incident CLOSED	Create new incident
Same service, different environment	Create new incident
Same service after duplicate window expires	Create new incident

Enterprise / Post-MVP evolution

Semantic similarity via embeddings, historical incident correlation, alert-storm clustering, cross-service dependency correlation, and AI-assisted duplicate detection. These can be added without changing the Feature 1 API surface, since duplicate detection remains encapsulated behind one service.

2.5 Data Model

Feature 1 owns three tables: `alert`, `incident`, `incident_timeline`.

```
alert (many) — incident_id (FK) -> incident (one)
incident (one) — incident_id (FK) -> incident_timeline (many)
```

Table: alert

System of record for every incoming alert, written to immediately regardless of whether it becomes a new incident or is linked to an existing one.

```

CREATE TABLE alert (
  alert_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  incident_id UUID NULL,
  external_alert_id VARCHAR(200),
  source_system VARCHAR(100) NOT NULL,
  alert_name VARCHAR(250) NOT NULL,
  alert_type VARCHAR(100),
  service_name VARCHAR(150) NOT NULL,
  application_name VARCHAR(150),
  environment VARCHAR(30) NOT NULL,
  severity VARCHAR(20) NOT NULL,
  metric_name VARCHAR(150),
  metric_value NUMERIC(12,2),
  threshold_value NUMERIC(12,2),
  alert_status VARCHAR(30),
  processing_status VARCHAR(30) NOT NULL,
  duplicate_status VARCHAR(30) DEFAULT 'PENDING',
  correlation_key VARCHAR(200),
  fingerprint VARCHAR(200),
  received_at TIMESTAMP NOT NULL,
  processed_at TIMESTAMP,
  raw_payload JSONB NOT NULL,
  canonical_alert JSONB,
  validation_message TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_alert_incident FOREIGN KEY (incident_id) REFERENCES
incident(incident_id)
);

```

Value set	Values
processing_status	RECEIVED → VALIDATED → NORMALIZED → ENRICHED → CORRELATED → INCIDENT_LINKED
duplicate_status	PENDING, UNIQUE, DUPLICATE

Table: incident

A production incident under investigation; one incident may be associated with multiple alerts.

```

CREATE TABLE incident (
  incident_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  incident_number VARCHAR(30) UNIQUE NOT NULL,
  title VARCHAR(300) NOT NULL,
  description TEXT,
  service_name VARCHAR(150) NOT NULL,
  application_name VARCHAR(150),
  environment VARCHAR(30) NOT NULL,
  severity VARCHAR(20) NOT NULL,
  priority VARCHAR(20),
  category VARCHAR(50),
  status VARCHAR(30) NOT NULL,
  current_feature VARCHAR(50),
  fingerprint VARCHAR(128) NOT NULL,
  owner_team VARCHAR(150),
  owner_email VARCHAR(200),
  opened_at TIMESTAMP NOT NULL,
  acknowledged_at TIMESTAMP,
  resolved_at TIMESTAMP,
  closed_at TIMESTAMP,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_incident_fingerprint ON incident (fingerprint);

```

status: OPEN → INVESTIGATING → RCA_COMPLETED → REMEDIATION_PENDING → RESOLVED → CLOSED. current_feature tracks which AIR feature (2–6) currently owns the incident.

Table: incident_timeline

Append-only audit trail of every significant incident lifecycle event, shared by all six features.

```

CREATE TABLE incident_timeline (
  timeline_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  incident_id UUID NOT NULL,
  feature_name VARCHAR(100),
  event_name VARCHAR(150),
  event_type VARCHAR(50),
  event_summary TEXT,
  actor_type VARCHAR(30),
  actor_name VARCHAR(100),
  metadata JSONB,
  event_timestamp TIMESTAMP NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_timeline_incident FOREIGN KEY (incident_id) REFERENCES
incident(incident_id)
);

```

2.6 API Specification

Method	Endpoint	Purpose
POST	/api/v1/alerts	Receive alert from monitoring tool
GET	/api/v1/alerts/{alertId}	Get alert details
GET	/api/v1/incidents/{incidentId}	Get incident details
GET	/api/v1/incidents	Search incidents (status, severity, service, environment, page, size)
GET	/api/v1/incidents/{id}/timeline	Incident timeline
POST	/api/v1/incidents/{id}/acknowledge	Acknowledge incident
POST	/api/v1/incidents/{id}/resolve	Resolve incident
POST	/api/v1/incidents/{id}/close	Close incident

POST /api/v1/alerts

Headers: Authorization: Bearer <token>, Content-Type: application/json, X-Correlation-Id (required), X-Source-System (required).

Request

```
{ "externalAlertId": "ALT-001", "alertName": "High HTTP 5XX", "alertType":  
"APPLICATION",  
  "serviceName": "payment-api", "applicationName": "Payment", "environment":  
"PROD",  
  "severity": "CRITICAL", "metricName": "http_5xx_rate", "metricValue": 18.6,  
  "thresholdValue": 5,  
  "status": "FIRING", "timestamp": "2026-08-05T10:15:42Z", "payload": {} }
```

Success – 202 Accepted

```
{ "alertId": "4b5dfe62-...", "incidentId": "4d3c50b5-...", "incidentNumber": "INC-  
20260805-000001",  
  "processingStatus": "INCIDENT_LINKED", "duplicateStatus": "UNIQUE", "message":  
"Alert accepted successfully." }
```

Error – 400 Bad Request

```
{ "code": "INVALID_ALERT", "message": "Service Name is mandatory." }
```

Validation: externalAlertId, alertName, serviceName mandatory; environment ∈ {PROD, UAT, DEV}; severity ∈ {CRITICAL, HIGH, MEDIUM, LOW}; metricValue/thresholdValue numeric; payload valid JSON.

Backend processing: Persist → Validate → Normalize → Context Enrichment → Duplicate Detection → Create/Link Incident → Timeline → Return Response.

Internal Service Design

Validation, normalization, enrichment, duplicate detection, and incident creation shall not be exposed as separate REST APIs. These are internal operations orchestrated by a single AlertIntakeService, which delegates to single-responsibility collaborators:

AlertValidationService, AlertNormalizationService, ContextEnrichmentService, DuplicateDetectionService, IncidentService, TimelineService, and a PlannerClient that hands off to Feature 2.

Handoff to Feature 2

On completion, Feature 1 shall invoke the Investigation Planner via an internal REST client, passing only the identifiers — the Planner reloads full context from PostgreSQL:

```
POST /internal/api/v1/planner/start
{ "incidentId": "4d3c50b5-c8e2-4f53-a7e8-fec6c14903af", "incidentNumber": "INC-20260805-000001" }
```

Enterprise / Post-MVP evolution

Replace the synchronous internal REST call with a Kafka event (`incident.investigation.requested`) so Feature 1 and the AI service scale and fail independently.

2.7 Cross-Cutting Runtime Artifacts

Three artifacts shall be frozen before implementation begins, as they define runtime behavior for the whole platform, not just Feature 1.

1 — Exception Handling Standard

Every API shall return the same error response contract via one global exception handler (`@RestControllerAdvice` in Spring Boot), so the UI can parse errors uniformly.

```
{ "timestamp": "2026-08-05T10:15:42Z", "status": 400, "error":
"VALIDATION_ERROR",
  "message": "serviceName is mandatory.", "path": "/api/v1/alerts",
  "correlationId": "CORR-123456" }
```

Category	HTTP	Example
Validation	400	Missing Service Name
Authentication	401	Invalid JWT
Authorization	403	User not allowed
Resource Not Found	404	Incident not found
Conflict	409	Duplicate incident
Business Rule	422	Invalid status transition
Internal Error	500	Database failure
External System	502	Planner unavailable

2 — State Machines

Every transition shall go through an explicit `transitionTo(...)` call, never a raw `setStatus(...)`, so invalid transitions raise a `BusinessRuleException`.

```
Alert:    RECEIVED → VALIDATED → NORMALIZED → ENRICHED → CORRELATED →
INCIDENT_LINKED | VALIDATION_FAILED
Incident: OPEN → INVESTIGATING → RCA_COMPLETED → REMEDIATION_PENDING →
RESOLVED → CLOSED
```

Feature	Owns transition to
Feature 1	OPEN
Feature 2	INVESTIGATING
Feature 5	RCA_COMPLETED
Feature 6	REMEDIATION_PENDING
Operator	RESOLVED, CLOSED

Invalid transitions (e.g. `CLOSED → OPEN`, `RCA_COMPLETED → INVESTIGATING`, `RECEIVED → INCIDENT_LINKED`) shall always throw `BusinessRuleException`.

3 — Backend Sequence & Responsibilities

```
AlertController → AlertIntakeService → { ValidationService, NormalizationService,
    EnrichmentService, DuplicateDetectionService, IncidentService, TimelineService,
    PlannerClient }
```

Component	Responsibility
AlertIntakeService	Workflow orchestration only — contains no business logic itself
ValidationService	Validate request
NormalizationService	Create the Canonical Alert
EnrichmentService	Add business metadata
DuplicateDetectionService	Determine incident correlation
IncidentService	Create or link incident
PlannerClient	Invoke Feature 2

2.8 Context Enrichment

The normalized alert shall be enriched from enterprise systems before AI investigation begins:

Source	Information
CMDB	Application owner
Service Registry	Service metadata
Kubernetes	Cluster, namespace, pods
GitHub	Last deployment
ServiceNow	Existing incidents
PagerDuty	On-call engineer
Architecture Repository	Dependencies

2.9 Design Decisions

Decision	Rationale
No LLM in Alert Intake	Deterministic processing is faster, cheaper, and more reliable.
Canonical Alert Model	Decouples downstream services from vendor-specific alert formats.
Asynchronous processing (202 Accepted)	Returns immediately and prevents monitoring-tool webhook timeouts.
Kafka event bus	Decouples alert ingestion from AI investigation, improves scalability.
Context enrichment before AI	Planner Agent starts with complete operational context.
Deterministic duplicate detection	Predictable, fast, explainable; prevents redundant investigations during alert storms.
Fingerprint stored on incident	Enables fast indexed lookups without joining back to the alert table.
Store raw and normalized payloads	Supports auditing, replay, and troubleshooting.
JSONB for payload storage	Preserves vendor-specific fields while enabling flexible querying.

2.10 Deliverables

Deliverable	Description
Validated Alert	Incoming alert authenticated and validated
Canonical Alert	Vendor payload converted into AIR's standard model
Enriched Incident Context	Service metadata, ownership, deployment, operational context
Incident Record	New incident created, or existing incident updated
Alert Record	Original and normalized alert persisted
Timeline Entry	Initial incident event recorded for auditing
Investigation Event	Kafka event published to trigger AI investigation
LangGraph Initial State	Initial context prepared for the Planner Agent

3. Feature 2 — Investigation Planner Agent

3.1 Objective & Scope

The Investigation Planner is the first AI-powered component in AIR. It shall convert an incident into an executable investigation strategy: classify the incident, determine which specialist agents should run, identify the evidence required, and decide which investigations can execute in parallel. Its output is an Investigation Plan, not a root cause — the Planner does not investigate and does not access monitoring tools directly.

In scope
Receive investigation request · load incident context · retrieve enterprise metadata · analyze alert semantics · classify incident category · determine investigation strategy · select required specialist agents · generate investigation tasks · initialize LangGraph state · persist investigation plan · publish investigation tasks to Feature 3.

3.2 Architecture Principles

Principle	Rule
Planner is an orchestrator	The Planner shall never call Splunk, Prometheus, or any tool directly. It generates an Investigation Plan; specialist agents (Feature 3) execute it.
One responsibility	The Planner produces only the Investigation Strategy, Evidence Requests, and Investigation Priority — nothing else.
Stateless reasoning	The Planner shall hold no internal memory; all state flows through LangGraph state exclusively.
Structured output only	The Planner never returns free text; every response is validated against a Pydantic schema.

3.3 LangGraph Design

START → Load Investigation → Planner Agent → Validate Output → Persist Output → Update Status → END (no conditional branches beyond success/failure are required for the MVP)
--

```
class InvestigationState(TypedDict):
    investigation_id: str
    incident: dict
    planner_output: dict | None
    status: str
    messages: list
```

State shall hold only data required by the Planner — never raw logs, metrics, or traces; those are scoped to Feature 3.

Node	Responsibility
Load Investigation	Read the investigation, incident, and alert from PostgreSQL
Planner Agent	Call the LLM to generate the investigation strategy
Validate Output	Validate against Pydantic; retry up to 2 times on failure
Persist Output	Update investigation, insert investigation_event and llm_execution
Update Status	Transition investigation status to EVIDENCE_COLLECTION; graph ends, Feature 3 begins

3.4 Prompt Design

Prompt construction shall be delegated to a dedicated PromptBuilder, fed by a ContextBuilder that assembles business context from repositories — the Planner Node itself never concatenates strings or calls the OpenAI SDK directly. Every model call is routed through one shared LLMService.

```
Investigation → Context Builder → Prompt Builder → LLM Service → OpenAI GPT-5.5
→ Structured Output → Pydantic Validation
```

System Prompt

You are an Enterprise Site Reliability Engineering (SRE) Investigation Planner.

Your responsibility is to analyze an incident and create an investigation strategy.

You are NOT responsible for finding the root cause.

You are NOT responsible for recommending remediation.

Your responsibilities are:

1. Classify the incident.
2. Determine the investigation strategy.
3. Determine which specialist agents should be executed.
4. Identify the evidence required.
5. Identify which investigations can execute in parallel.

Always return valid JSON that matches the provided schema.

Never invent evidence. Never assume the root cause.

If insufficient information exists, request additional evidence rather than guessing.

Structured Output Model

```
class PlannerOutput(BaseModel):
    plannerVersion: str
    investigationType: str
    priority: str
    parallelAgents: list[str]
    requiredEvidence: list[str]
    hypotheses: list[str]
    reasoning: str
```

```
{ "plannerVersion": "1.0", "investigationType": "Application", "priority":
"Critical",
  "parallelAgents": ["LogsAgent","MetricsAgent","DeploymentAgent"],
  "requiredEvidence": ["Application Logs","HTTP Error Logs","Deployment
History","CPU Metrics"],
  "hypotheses": ["Deployment Regression","Configuration Drift"],
  "reasoning": "The alert occurred shortly after a deployment. Evidence should
focus on deployment
    changes, application logs, and service health before considering
infrastructure issues." }
```

This response does not conclude "deployment caused the issue" — it states that deployment should be investigated first because it is a plausible hypothesis. That distinction shall be preserved throughout the platform: only Feature 4 (RCA) is permitted to draw conclusions.

The response shall be requested via **OpenAI Structured Outputs** against the `PlannerOutput` schema, never via a free-form "return JSON" instruction, and validated with Pydantic before persistence. One or two few-shot examples may be included in the prompt; more than that increases token usage without materially improving quality.

3.5 Data Model

`investigation` is the aggregate root of Feature 2; the planner output is stored as JSONB rather than normalized into a hypothesis/task table hierarchy, to keep the MVP schema simple and avoid premature normalization.

```
CREATE TABLE investigation (  
  investigation_id UUID PRIMARY KEY,  
  incident_id UUID NOT NULL,  
  status VARCHAR(40) NOT NULL,  
  planner_output JSONB,  
  started_at TIMESTAMP,  
  completed_at TIMESTAMP,  
  created_at TIMESTAMP NOT NULL,  
  updated_at TIMESTAMP NOT NULL  
);  
CREATE INDEX idx_investigation_incident ON investigation(incident_id);  
CREATE INDEX idx_investigation_status ON investigation(status);
```

`status` $\in$ {NEW, PLANNING, EVIDENCE_COLLECTION, RCA_ANALYSIS, RECOMMENDATION, COMPLETED, FAILED}.

```
CREATE TABLE investigation_event (  
  event_id UUID PRIMARY KEY,  
  investigation_id UUID NOT NULL,  
  event_type VARCHAR(100),  
  agent_name VARCHAR(100),  
  message TEXT,  
  event_time TIMESTAMP NOT NULL  
);
```

Recommended refinement

Replace the free-form `event_type/message` pair with structured `node_name`, `agent_name`, `event_status`, and `duration_ms` columns so the same table serves as a live execution monitor across all six features, not just an audit log.

```
CREATE TABLE llm_execution (  
  execution_id UUID PRIMARY KEY,  
  investigation_id UUID NOT NULL,  
  agent_name VARCHAR(100),  
  model_name VARCHAR(100),  
  prompt_version VARCHAR(30),  
  temperature DECIMAL(3,2),  
  input_tokens INTEGER,  
  output_tokens INTEGER,  
  total_tokens INTEGER,  
  latency_ms INTEGER,  
  estimated_cost DECIMAL(12,6),  
  status VARCHAR(20),  
  created_at TIMESTAMP  
);  
CREATE INDEX idx_llm_execution ON llm_execution(investigation_id);
```

This table is shared by every AI-enabled feature (Planner, Logs Agent, RCA, Resolution Planning, Incident Report) and is the sole source for cost monitoring, replay, and prompt optimization — it shall never be skipped for an LLM call.

3.6 API Specification

React UI → Spring Boot API Gateway (public) → Python FastAPI (internal) →
LangGraph Planner Agent

Method	Endpoint	Consumer	Description
POST	/api/v1/investigations	Alert Intake	Creates investigation, starts planning
GET	/api/v1/investigations/{id}	UI	Investigation status
GET	/api/v1/investigations/{id}/plan	UI	Investigation plan (planner output)
GET	/api/v1/investigations/{id}/events	UI	Planner events / timeline
POST	/api/v1/investigations/{id}/retry	UI/Admin	Retries planner (only on FAILED or PLANNING_TIMEOUT)
POST	/api/v1/investigations/{id}/cancel	UI/Admin	Cancels investigation (not COMPLETED)
POST	/internal/v1/planner/execute	Spring Boot only	Executes the LangGraph planner
GET	/internal/v1/health	Kubernetes	Database + OpenAI connectivity

Create Investigation is asynchronous (202 Accepted). Internal calls pass only the investigationId; the Python service reloads the complete investigation and incident context from PostgreSQL — this prevents Spring Boot and Python from maintaining two divergent copies of investigation context.

POST /api/v1/investigations

Request: { "incidentId": "INC-10001" }

202 Accepted

{ "investigationId": "INV-1001", "incidentId": "INC-10001", "status": "PLANNING", "message": "Investigation created successfully." }

Exceptions: 404 Incident Not Found · 409 Investigation Already Exists · 400 Invalid Request · 500 Internal Error

Status	Usage
200 / 202	Success (sync) / accepted (async)
400 / 404 / 409	Validation / not found / invalid state transition
422	Planner output validation failed
429 / 502 / 504	Rate limiting / AI service unavailable / planner timeout

API best practices: resource-oriented URLs, one shared error contract, correlation IDs propagated end to end, an Idempotency-Key header on POST /investigations, /api/v1 and /internal/v1 prefixing, and public APIs that never expose prompts, model names, or raw LLM responses.

MVP SCOPE

For the MVP, replace the synchronous Spring Boot → Python call with an InvestigationRequested Kafka event consumed by the Python AI service. This removes tight coupling, keeps the public API responsive during multi-second AI processing, and matches how

Feature 3 evidence collection is triggered.

3.7 Component Architecture

Spring Boot Gateway → Python FastAPI → InvestigationController → InvestigationService → PlannerService → { ContextBuilder, PromptBuilder, LLMService } → OpenAI GPT-5.5 → InvestigationRepository → PostgreSQL

Class	Responsibility
InvestigationController	Expose internal REST APIs; never contains business logic
InvestigationService	Coordinates investigation lifecycle (create, start, update_status, complete, fail)
PlannerService	Owns the full planning workflow: load context, build prompt, call LLM, validate, persist
ContextBuilder	Converts database entities into planner context; contains no prompt logic
PromptBuilder	Loads the prompt template and merges system prompt, user prompt, and context
LLMService	Sole owner of the OpenAI SDK: retry, structured output, logging, cost/token metrics
InvestigationRepository / EventRepository / LLMExecutionRepository	Own persistence; the Planner never executes SQL directly

Package Structure

```
air-agent-app/  
  app.py  
  api/          investigation_controller.py, planner_controller.py,  
health_controller.py  
  graph/        investigation_graph.py, graph_factory.py, state.py  
  agents/       planner_agent.py  
  services/     investigation_service.py, planner_service.py, llm_service.py,  
                prompt_builder.py, context_builder.py  
  repositories/ investigation_repository.py, event_repository.py,  
llm_execution_repository.py  
  models/       entity/, dto/, request/, response/  
  prompts/      planner.md  
  config/       settings.py, logging.py  
  utils/  
  tests/
```

Failure Handling

Error	Retry	Action
Database unavailable	No	Investigation FAILED
OpenAI timeout	Yes (2 attempts)	Retry, then FAILED
Invalid structured output	Yes	Retry with schema reminder
Repository error	No	Rollback
Investigation not found	No	Return 404

3.8 Definition of Done

A Feature 2 implementation is complete when all of the following hold:

- Investigation can be created and the planner executes successfully through LangGraph.
- Context is built from persisted data; the prompt is generated from versioned templates.
- OpenAI returns Structured Output that passes Pydantic validation.
- `planner_output` is stored on `investigation`; status transitions to `EVIDENCE_COLLECTION`.
- `investigation_event` records *Planner Started* and *Planner Completed*.
- `llm_execution` captures token usage, latency, and cost for every call.
- Unit test coverage $\geq 80\%$ for the service layer.

- OpenTelemetry metrics and structured JSON logs are emitted for every node.
- The health endpoint reports database and OpenAI connectivity.

3.9 Recommended Build Order

#	Component	Depends on
1	SQLAlchemy entity models	Database schema
2	Repository layer	Entity models
3	Pydantic request/response models	API contracts
4	Context Builder	Repositories
5	Prompt Builder	Prompt templates
6	LLM Service	OpenAI SDK
7	Planner Service	Context Builder + Prompt Builder + LLM Service
8	LangGraph state & Planner node	Planner Service
9	FastAPI controllers	Planner Service
10	Unit & integration tests	All components

Coding standards: one class = one responsibility; repositories contain no business logic; prompt text is never embedded in Python source; controllers never call OpenAI directly; LangGraph nodes orchestrate, they do not implement business logic; all AI responses are Pydantic-validated before persistence; dependencies are injected via constructors, never global singletons.

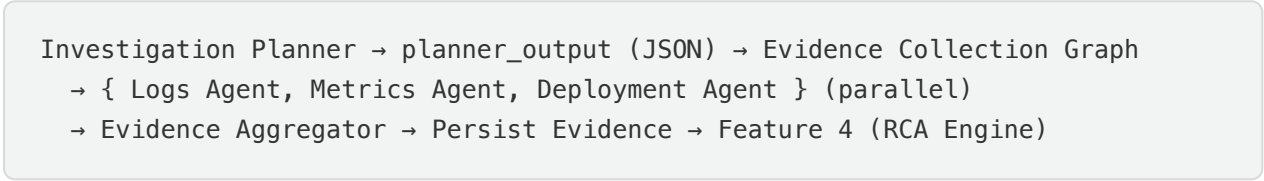
4. Feature 3 — Evidence Collection Framework

4.1 Objective

The Evidence Collection Framework shall execute the specialist agents named in the Investigation Plan in parallel, collect evidence from multiple enterprise systems, normalize it into a common format, persist it, and return it to LangGraph. It shall not determine the root cause, rank hypotheses, or recommend remediation — those remain the responsibility of Features 4 and 5.

Unlike Feature 2, Feature 3 is not a single agent: it is a collection of specialist agents coordinated by LangGraph, because a production incident rarely has a single source of truth. A payment API returning HTTP 500s cannot be explained from logs alone; the platform needs correlated input from logs, metrics, Kubernetes, deployments, and traces, each behind its own specialist agent.

4.2 Overall Architecture



Agent Responsibilities

Agent	Responsible for	Never
LogsAgent	Application/exception logs, error patterns, stack traces, log frequency	Calculates RCA or recommends fixes
MetricsAgent	CPU, memory, JVM, HTTP, database metrics, response time	Interprets deployments or reads logs
DeploymentAgent	Deployment history, Git commits, release versions, rollbacks, config changes	Analyzes runtime logs

Enterprise / Post-MVP agents (not required for MVP)
Trace Agent, Kubernetes Agent, Database Agent, Network Agent, Security Agent, Cloud Agent, ServiceNow Agent.

4.3 LangGraph Subgraph & Parallel Execution

Feature 3 shall run as a LangGraph subgraph. The Planner has already decided which agents to run (`planner_output.parallelAgents`); LangGraph's role is execution, not agent selection.

```
START → Read planner_output → Dispatch Parallel Agents
    → { LogsNode, MetricsNode, DeploymentNode } (concurrent)
    → AggregateEvidenceNode → CorrelationEngineNode → PersistCorrelationNode
    → Update Investigation Status → END
```

Executing agents sequentially would sum their durations; running them concurrently bounds total investigation time to approximately the slowest agent's duration.

```
class InvestigationState(TypedDict):
    investigation_id: str
    planner_output: PlannerOutput
    evidence: list[Evidence]
    completed_agents: list[str]
    failed_agents: list[str]
    correlation_result: CorrelationResult | None
    status: str
```

Evidence Contract

Every agent, regardless of source system, shall return the same normalized structure:

```
{ "agent": "LogsAgent", "type": "LOG", "source": "Splunk",
  "summary": "NullPointerException observed after deployment.", "confidence":
  0.82,
  "content": { "exceptions": ["NullPointerException"] } }
```

Evidence Aggregator

The Aggregator waits for all parallel agents (or an agent-level timeout policy) and shall: (1) merge results, (2) normalize schema across agents, (3) remove/link duplicate observations, (4) sort by time, (5) validate completeness against the Planner's requested agent list, and (6) build an ordered evidence timeline. It shall never draw conclusions such as "deployment caused the failure" — that is reserved for the Correlation Engine and RCA Agent.

```
class AggregatedEvidence(BaseModel):
    investigation_id: str
    evidence: list[Evidence]
    missing_agents: list[str]
    duplicate_count: int
    timeline: list[TimelineEvent]
```

Error Isolation

If one agent fails (e.g. Deployment Agent), the investigation shall continue with the successful agents; only the failed agent is marked as failed. Each agent status $\in \{PENDING, RUNNING, COMPLETED, FAILED, SKIPPED\}$ is recorded in `investigation_event`. Agents shall never call each other directly — all coordination happens through LangGraph state (Logs Agent $\rightarrow$ State $\rightarrow$ Aggregator $\rightarrow$ State $\rightarrow$ RCA).

Class Design

```
BaseEvidenceAgent (abstract: collect(), normalize(), summarize(), execute())
  ↑— LogsAgent, MetricsAgent, DeploymentAgent (override collection logic only)
```

4.4 Evidence Intelligence Engine

After aggregation, evidence is still an unrelated bag of facts. The Evidence Intelligence Engine (renamed from "Correlation Engine") shall transform that bag of facts into structured **investigation knowledge** — timeline ordering, groupings, and temporal/service relationships — without determining a root cause. This is a deterministic, rule-based step; it does not call an LLM.

```
Feature 2 Planner → Feature 3 Evidence Collection → Evidence Consolidator
(facts only)
  → Evidence Intelligence Engine → InvestigationKnowledge (facts +
relationships) → Feature 4 RCA Agent
```

Layer	Says
Evidence Consolidator	"Here are all the facts."
Evidence Intelligence Engine	"Here is how the facts relate to one another."
RCA Agent (Feature 4)	"Given these relationships, what is the most likely root cause?"

Example output (`InvestigationKnowledge`), built from three independent Logs/Metrics/Deployment observations:

```
{ "investigationId": "INV-1001",
  "timeline": [
    { "time": "09:54:00Z", "type": "DEPLOYMENT", "event": "Version 2.8.1 deployed"
  },
    { "time": "10:05:00Z", "type": "ALERT", "event": "HTTP 5XX threshold exceeded"
  },
    { "time": "10:12:15Z", "type": "APPLICATION", "event": "482
NullPointerException detected" },
    { "time": "10:13:00Z", "type": "INFRASTRUCTURE", "event": "CPU stable at 28%"
  } ],
  "evidenceGroups": [ { "group": "Application", "evidence":
["NullPointerException","HTTP 5XX"] },
    { "group": "Deployment", "evidence": ["Version 2.8.1","Commit abc123"] },
    { "group": "Infrastructure", "evidence": ["CPU","Memory"] } ],
  "relationships": [ { "type": "TEMPORAL", "source": "Deployment", "target":
"Application Exception",
    "confidence": 0.91, "reason": "Application exceptions started 18 minutes
after deployment." } ],
  "contradictions": [], "healthyComponents": ["CPU","Memory"],
  "affectedComponents": ["PaymentProcessor","payment-api"],
  "investigationSummary": { "applicationHealth": "UNHEALTHY",
  "infrastructureHealth": "HEALTHY",
    "deploymentDetected": true, "relationshipCount": 2, "missingEvidence": [] } }
```

```
class InvestigationKnowledge(BaseModel):
    timeline: list[TimelineEvent]
    evidence_groups: list[EvidenceGroup]
    relationships: list[Relationship]
    healthy_components: list[str]
    affected_components: list[str]
    contradictions: list[Contradiction]
    investigation_summary: InvestigationSummary
```

Sending InvestigationKnowledge instead of raw evidence to the RCA Agent reduces prompt size to roughly 20–30% of the original evidence volume — materially cheaper and more deterministic for GPT to reason over. This creates a three-layer architecture that shall be treated as a platform standard:

Evidence (facts collected from tools) → **Knowledge** (relationships derived deterministically) → **Reasoning** (AI decision from that knowledge, Feature 4 only).

4.5 Data Model

PostgreSQL shall store normalized evidence summaries and source references, never the entire telemetry payload (e.g. 50,000 Splunk log lines or 10,000 Prometheus samples remain in Splunk/Prometheus). This matches standard enterprise AIOps storage practice.

```
CREATE TABLE evidence (  
  evidence_id UUID PRIMARY KEY,  
  investigation_id UUID NOT NULL,  
  agent_name VARCHAR(100) NOT NULL,  
  evidence_type VARCHAR(50) NOT NULL,  
  source_system VARCHAR(100) NOT NULL,  
  title VARCHAR(300),  
  summary TEXT,  
  confidence DECIMAL(4,2),  
  findings JSONB,  
  collected_at TIMESTAMP NOT NULL,  
  created_at TIMESTAMP NOT NULL  
);  
CREATE INDEX idx_evidence_investigation ON evidence(investigation_id);  
CREATE INDEX idx_evidence_type ON evidence(evidence_type);  
  
CREATE TABLE evidence_reference (  
  reference_id UUID PRIMARY KEY,  
  evidence_id UUID NOT NULL,  
  reference_type VARCHAR(50),  
  reference_value TEXT,  
  created_at TIMESTAMP  
);  
CREATE INDEX idx_reference_evidence ON evidence_reference(evidence_id);
```

evidence_type ∈ {LOG, METRIC, DEPLOYMENT} for MVP (TRACE, DATABASE, NETWORK, SECURITY, CLOUD are Enterprise). evidence_reference is a separate table because one evidence record may carry several references — a Splunk search URL, a GitHub commit SHA, and a Jaeger trace ID simultaneously.

```
findings example (Logs Agent):  
{ "exception": "NullPointerException", "class": "PaymentProcessor", "count": 482,  
  "firstSeen": "10:12", "lastSeen": "10:15" }
```

4.6 API Specification

No public API starts Evidence Collection — it is triggered internally by the Planner. Only Spring Boot exposes public APIs.

Method	Endpoint	Description
GET	/api/v1/investigations/{id}/evidence	All collected evidence (public)
GET	/api/v1/evidence/{evidenceId}	One evidence record with findings + references (public)
POST	/internal/v1/evidence/execute	Starts the evidence collection graph (internal, called by Planner)
GET	/internal/v1/evidence/{id}/status	Orchestration/monitoring status, not for UI
POST	/internal/v1/evidence/{id}/retry	Retries failed agents only (or all, if none specified)
GET	/internal/v1/evidence/health	Database, OpenAI, and tool adapter health

POST /internal/v1/evidence/execute

Request: { "investigationId": "INV-1001" }

202 Accepted – { "status": "RUNNING", "graph": "EvidenceCollectionGraph", "message": "Evidence collection started." }

Exceptions: 404 Investigation Not Found · 409 Planner Not Completed · 422 Invalid Planner Output

POST /internal/v1/evidence/execute shall be idempotent: if evidence collection is already running for the investigation, the endpoint returns the current execution state rather than starting a second LangGraph execution.

API type	Authentication
Public APIs	OAuth2/JWT via Spring Boot
Internal APIs	mTLS or service-to-service JWT
Health APIs	Kubernetes network policy (internal only)

The Python AI service shall never be exposed directly to end users. Public evidence endpoints shall return normalized structures only — never raw telemetry.

MVP SCOPE

Keep Spring Boot as the API façade and the Python AI service as an internal orchestration engine. An even stronger design replaces the synchronous internal call with an EvidenceCollectionRequested Kafka event, eliminating tight coupling and enabling horizontal scaling of AI workers.

4.7 Tool Integration Layer & Adapter Architecture

Evidence agents shall never interact with vendor APIs directly. A Tool Abstraction Layer shall hide authentication, API protocols, retry logic, rate limiting, and connection pooling behind a common adapter interface, so adding a new tool (e.g. Elastic) never requires changing agent code.

```
class ToolAdapter(ABC):
    def connect(self): ...
    def execute(self, request): ...
    def health(self): ...
    def close(self): ...
```

Tool	MVP Integration
Splunk / Prometheus / Datadog / Dynatrace	Direct REST API
GitHub	GraphQL / REST API
Kubernetes	Kubernetes Python Client
MCP-enabled tools	MCP Adapter, optional where available

This hybrid strategy gives maximum MVP stability while allowing gradual adoption of MCP as enterprise tooling matures. A Tool Registry shall resolve adapters by name (`registry.get("splunk")`) rather than hardcoded `if/else` branches, secrets shall come from Vault/Kubernetes Secrets/cloud secret managers (never hardcoded), retries follow exponential backoff with jitter (max 3 attempts) on 429/500/timeout only, and vendor errors shall be translated into standard AIR exceptions (`ToolRateLimitException`, `ToolTimeoutException`, `ToolAuthenticationException`) so agents only ever handle AIR's own exception types.

4.8 AI Agent Prompt Engineering

Not every agent requires an LLM. The AI Capability Matrix below shall gate LLM usage per feature to prevent unnecessary token spend:

Agent	Collect	Rule Engine	LLM Summary	LLM Reasoning
Planner	—	—	—	✓
Logs Agent	✓	✓	✓	—
Metrics Agent	✓	✓	—	—
Deployment Agent	✓	✓	—	—
Evidence Intelligence Engine	—	✓	—	—
RCA Agent	—	—	—	✓
Recommendation Agent	—	—	—	✓

The Logs Agent is the only evidence agent using an LLM in the MVP, to summarize chunked log excerpts:

System Prompt:

You are an Enterprise SRE Log Analysis Assistant. Your task is to summarize application logs.

Do not determine the root cause. Do not recommend remediation.

Extract only observable facts. Return valid structured JSON.

Expected Output:

```
{ "summary": "482 NullPointerExceptions detected.",  
  "exceptions": [ { "type": "NullPointerException", "count": 482 } ],  
  "affectedClasses": ["PaymentProcessor"], "patterns": ["Repeated exception  
after deployment"] }
```

Large log volumes (e.g. 20,000 lines) shall be chunked, summarized per chunk, then merged — never sent to the model in a single request. Guardrails required in every AI-enabled prompt across the platform: use only supplied evidence; never infer root cause or recommend fixes outside the owning feature; return structured JSON only; mark unknowns explicitly rather than fabricating data.

AIR-F004 Single GPT-5.5 reasoning call

5. Feature 4 — Root Cause Assessment

5.1 Objective

Feature 4 shall generate an explainable RootCauseAssessment from the InvestigationKnowledge produced in Feature 3, answering: What is the most likely root cause? Which evidence supports it? What is the confidence level? What evidence is missing? What alternative explanations should the operator consider? It shall not recommend remediation — that is Feature 5.

Feature 3 InvestigationKnowledge → Context Builder → Prompt Builder → OpenAI GPT-5.5
→ Structured Output Validation (Pydantic) → RootCauseAssessment → PostgreSQL
→ Feature 5

5.2 LangGraph Flow

START → Load InvestigationKnowledge → Build RCA Context → Generate Prompt
→ Call GPT-5.5 → Validate JSON → Persist RCA → END (only one AI node)

class RCAState(TypedDict):
 investigation_id: str
 investigation_knowledge: InvestigationKnowledge
 prompt: str
 root_cause_assessment: RootCauseAssessment | None
 status: str

Component	Responsibility	LLM
Context Builder	Build RCA context from InvestigationKnowledge	—
Prompt Builder	Generate prompt from a versioned template	—
RCA Agent	Perform reasoning	✓ GPT-5.5
Output Validator	Validate response against Pydantic schema	—
RCA Repository	Persist assessment	—

5.3 Prompt & Structured Output

System Prompt:

You are an expert Site Reliability Engineer (SRE) specializing in production incident analysis.

Your task is to determine the single most probable root cause using ONLY the supplied

Investigation Knowledge.

Rules:

1. Use only the provided evidence.
2. Never invent facts.
3. Do not recommend remediation.
4. Do not request additional data.
5. If evidence is insufficient, return UNKNOWN.
6. Explain your reasoning.
7. Always return valid JSON matching the provided schema.

```
class RootCause(BaseModel):
    category: str; title: str; description: str; confidence: float
class AlternativeCause(BaseModel):
    title: str; confidence: float
class RootCauseAssessment(BaseModel):
    root_cause: RootCause
    supporting_evidence: list[str]
    alternative_causes: list[AlternativeCause]
    missing_evidence: list[str]
    reasoning_summary: str
```

```
{ "root_cause": { "category": "Application", "title": "Deployment Regression",
    "description": "Deployment version 2.8.1 introduced a NullPointerException in
PaymentProcessor.",
    "confidence": 0.91 },
  "supporting_evidence": ["Deployment occurred before application failures.",
    "482 NullPointerExceptions detected.", "Infrastructure remained healthy."],
  "alternative_causes": [ { "title": "Configuration Drift", "confidence": 0.28 }
],
  "missing_evidence": [],
  "reasoning_summary": "The temporal relationship between deployment and
exceptions, combined
    with healthy infrastructure metrics, strongly indicates an application
regression." }
```

Guardrails: use only supplied InvestigationKnowledge; return exactly one primary root cause; a maximum of three alternative causes; confidence $\in [0.0, 1.0]$; if evidence is insufficient, return category: "Unknown", title: "UNKNOWN" with a low confidence rather than fabricating a conclusion; no remediation advice (reserved for Feature 5). Validation shall use OpenAI Structured Outputs bound to RootCauseAssessment — never manual JSON parsing.

5.4 Data Model

```
CREATE TABLE root_cause (  
  root_cause_id UUID PRIMARY KEY,  
  investigation_id UUID NOT NULL UNIQUE,  
  category VARCHAR(100),  
  title VARCHAR(300),  
  description TEXT,  
  confidence DECIMAL(5,2),  
  reasoning_summary TEXT,  
  supporting_evidence JSONB,  
  alternative_causes JSONB,  
  missing_evidence JSONB,  
  llm_execution_id UUID,  
  created_at TIMESTAMP,  
  updated_at TIMESTAMP  
);  
CREATE UNIQUE INDEX idx_root_cause_investigation ON root_cause(investigation_id);  
CREATE INDEX idx_root_cause_category ON root_cause(category);
```

One investigation maps to exactly one root cause assessment in the MVP (UNIQUE constraint). `supporting_evidence`, `alternative_causes`, and `missing_evidence` are stored as JSONB rather than normalized into separate `hypothesis/evidence_link` tables — this avoids joins, extra repositories, and extra migrations without losing information, and lets the schema absorb future evidence types (Network, Trace, Security, Database) with no migration.

5.5 API Specification

The MVP supports exactly one investigation, one RCA execution, and one final assessment — no versioning, approvals, or collaborative workflows. Only three APIs are required:

Method	Endpoint	Purpose	MVP
POST	/investigations/{id}/root-cause	Trigger RCA Agent	✓
GET	/investigations/{id}/root-cause	Retrieve RCA assessment	✓
GET	/investigations/{id}/root-cause/status	Check execution status	✓
POST	/investigations/{id}/root-cause/rerun	Re-execute after new evidence	Enterprise
GET	/investigations/{id}/root-cause/versions	Compare RCA versions	Enterprise
POST	/investigations/{id}/root-cause/review	Human review / approval	Enterprise

Controllers shall never call OpenAI directly (API Controller → RCA Service → Context Builder → Prompt Builder → LLM Service → Repository). Confidence scoring and explainability shall be provided directly by the LLM in the MVP (a returned confidence value plus a reasoning summary) rather than a separate deterministic scoring engine; the Enterprise edition can later replace LLM-generated confidence with a deterministic/LLM hybrid without changing the external API.

Error Handling

Error	Action
OpenAI timeout	Retry up to 3 times
Invalid schema	Retry once with an explicit schema reminder
Validation failure	Log and mark RCA as failed
No root cause identified	Return UNKNOWN with explanation instead of failing

MVP vs. Enterprise

Area	MVP	Enterprise
LLM calls	1	Multiple reasoning stages
Input	InvestigationKnowledge	+ Historical Incidents + Topology Graph
Confidence	LLM-generated	Deterministic + LLM hybrid
Development time	2–3 weeks	6–8 weeks

AIR-F005

Advisory only · no execution

6. Feature 5 — Resolution Planning Agent

6.1 Objective

Feature 5 shall transform a validated RootCauseAssessment into an actionable ResolutionPlan, answering: what should the operator do first, is rollback required, is manual intervention needed, which team should own the incident, and what are the risks. It shall not execute actions, roll back deployments, restart pods, or call external systems — execution remains a manual operator step in the MVP.

Feature 4 RootCauseAssessment → Recommendation Context Builder → Prompt Builder
→ OpenAI GPT-5.5 → Structured Output → Pydantic Validation → ResolutionPlan
→ PostgreSQL

#	Step	Example
1	Analyze root cause	Deployment Regression
2	Determine resolution strategy	Rollback latest deployment
3	Estimate risk	Low
4	Determine priority	Immediate
5	Identify owning team	Payment Platform Team
6	Suggest verification steps	Verify HTTP 200 responses after rollback

6.2 Prompt & Structured Output

System Prompt:
You are an expert Site Reliability Engineer.
Your task is to create a safe incident resolution plan.
Use only the supplied RootCauseAssessment. Do not invent evidence. Do not execute actions.
Return structured JSON.

```

class ResolutionStep(BaseModel):
    order: int; action: str; rationale: str
class ResolutionPlan(BaseModel):
    strategy: str; priority: str; risk: str; owner_team: str; estimated_effort:
str
    steps: list[ResolutionStep]
    verification_steps: list[str]
    rollback_required: bool
    escalation_required: bool
    summary: str

```

```

{ "strategy": "Rollback Deployment", "priority": "P1", "risk": "LOW",
  "owner_team": "Payment Platform",
  "estimated_effort": "15 minutes", "rollback_required": true,
  "escalation_required": false,
  "steps": [ { "order": 1, "action": "Rollback deployment version 2.8.1",
    "rationale": "Application failures began immediately after deployment." },
    { "order": 2, "action": "Monitor HTTP 5XX errors", "rationale": "Verify
rollback effectiveness." } ],
  "verification_steps": ["HTTP 5XX returns below threshold.", "No new
NullPointerExceptions observed."],
  "summary": "Rollback the latest deployment and monitor application health." }

```

Guardrails: use only the supplied RCA; never execute actions (MVP remains advisory); maximum five steps; always include verification steps; do not recommend actions unsupported by the evidence.

6.3 Data Model

```
CREATE TABLE resolution_plan (  
  resolution_plan_id UUID PRIMARY KEY,  
  investigation_id UUID NOT NULL UNIQUE,  
  strategy VARCHAR(200),  
  priority VARCHAR(20),  
  risk VARCHAR(20),  
  owner_team VARCHAR(200),  
  estimated_effort VARCHAR(100),  
  rollback_required BOOLEAN,  
  escalation_required BOOLEAN,  
  steps JSONB,  
  verification_steps JSONB,  
  summary TEXT,  
  llm_execution_id UUID,  
  created_at TIMESTAMP,  
  updated_at TIMESTAMP  
);
```

As with root_cause, one investigation maps to exactly one resolution plan in the MVP — no editing, no approvals, no execution.

6.4 API Specification

Method	Endpoint	Purpose
POST	/investigations/{id}/resolution-plan	Execute the Recommendation Agent
GET	/investigations/{id}/resolution-plan	Retrieve plan by investigation
GET	/resolution-plan/{id}	Retrieve plan by ID

6.5 LangGraph & State

```
START → Load RootCauseAssessment → Build Prompt → GPT-5.5 → Validate JSON  
  → Persist ResolutionPlan → END  
  
Incident state: RCA_COMPLETED → RESOLUTION_PLANNING → RESOLUTION_COMPLETED →  
CLOSED (manual)
```

6.6 MVP vs. Enterprise

Capability	MVP	Enterprise
Resolution Plan (advisory)	✓	✓
Automated runbooks	—	✓
Kubernetes actions	—	✓
GitHub rollback	—	✓
ServiceNow change creation	—	✓
Auto remediation	—	✓
Human approval workflow	—	✓

MVP SCOPE

The MVP shall stop at advisory recommendations. No action in Features 1–5 executes a change against a production system.

AIR-F006 Closes the MVP lifecycle

7. Feature 6 — AI Incident Report & Knowledge Base

7.1 Objective

Feature 6 closes the MVP lifecycle by turning the Root Cause Assessment and Resolution Plan into a structured, human-readable incident report and organizational knowledge.

Capability	MVP	Enterprise
Incident Summary	✓	Enhanced
AI Post-Mortem Report	✓	Enhanced
Lessons Learned	✓	Enhanced
Knowledge Base Storage	✓	Vector search over past incidents

7.2 LLM Prompt & Output

System Prompt:

You are an expert Incident Manager. Generate a professional incident report. Use only supplied information. Do not invent facts. Return structured JSON.

Input: RootCauseAssessment + ResolutionPlan

Output: { "executiveSummary": "...", "timeline": [...], "lessonsLearned": [...], "preventiveActions": [...] }

This is documentation generation, not reasoning — the report shall not introduce any conclusion not already present in the RCA or Resolution Plan. Only one LLM call is required.

7.3 Data Model (MVP)

```
CREATE TABLE incident_report (  
  incident_report_id UUID PRIMARY KEY,  
  investigation_id UUID NOT NULL,  
  executive_summary TEXT,  
  timeline JSONB,  
  root_cause_summary TEXT,  
  resolution_summary TEXT,  
  lessons_learned JSONB,  
  preventive_actions JSONB,  
  created_at TIMESTAMP  
);
```

7.4 API Specification (MVP)

Method	Endpoint	Purpose
POST	/incident-report	Generate report
GET	/incident-report/{id}	View report
GET	/investigation/{id}/incident-report	Report by investigation

MVP SCOPE — explicit boundary

For the MVP: store `incident_report` only in PostgreSQL. Do NOT implement `knowledge_document` or `embedding_job`. Do NOT integrate a vector database. Keep the RCA Agent focused solely on the current investigation — everything below this line is Enterprise roadmap, not MVP build scope.

7.5 Enterprise Evolution — Knowledge Base

The Enterprise roadmap adds two tables to track publishing of operational artifacts to a semantic knowledge store, so that a future Planner can ask "have we seen this before?" and reuse a prior investigation plan.

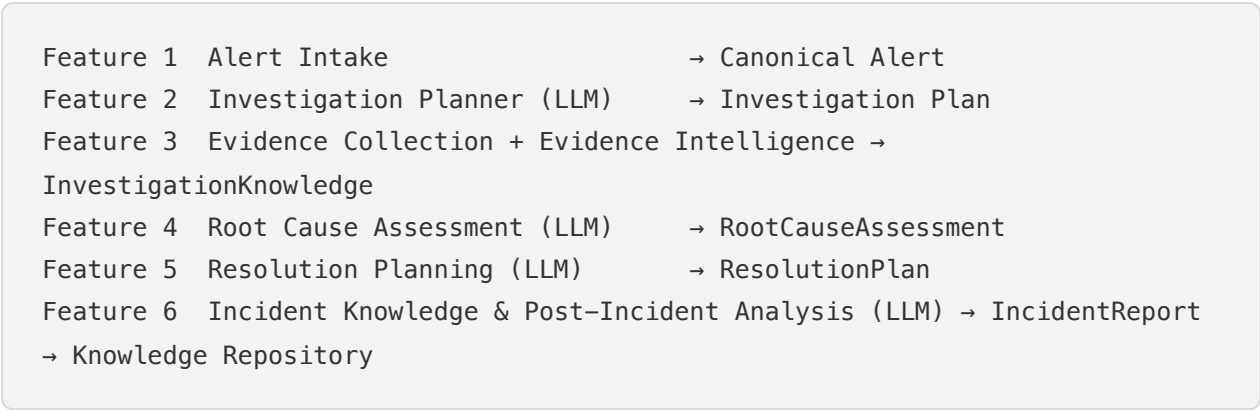
```
CREATE TABLE knowledge_document (  
  knowledge_document_id UUID PRIMARY KEY, investigation_id UUID NOT NULL,  
  document_type VARCHAR(50), source_record_id UUID NOT NULL, title VARCHAR(300),  
  summary TEXT,  
  tags JSONB, metadata JSONB, embedding_status VARCHAR(30), vector_document_id  
  VARCHAR(200),  
  created_at TIMESTAMP, updated_at TIMESTAMP  
);  
CREATE TABLE embedding_job (  
  embedding_job_id UUID PRIMARY KEY, knowledge_document_id UUID NOT NULL,  
  model_name VARCHAR(100), status VARCHAR(30), retry_count INTEGER, error_message  
  TEXT,  
  started_at TIMESTAMP, completed_at TIMESTAMP  
);
```

PostgreSQL remains the source of truth; if using Pinecone/Weaviate/Milvus, store only the vector document ID, status, and metadata in Postgres — not the embedding vector itself. If using PGVector, the vector may be stored in the same database. Recommended MVP→Enterprise search contract:

```
POST /api/v1/knowledge/search  Request: { "query": "Payment API  
NullPointerException after deployment", "topK": 3 }  
Response: { "results": [ { "investigationId": "INV-452", "similarity": 0.96,  
  "rootCause": "Deployment Regression", "resolution": "Rollback deployment" } ] }
```

7.6 Complete MVP Architecture

With Feature 6 complete, the MVP delivers the full closed-loop investigation pipeline:



8. Appendix — Cross-Cutting Design Decisions & Roadmap

8.1 Platform-Wide Design Standards

These decisions recur across all six features and shall be enforced consistently rather than re-litigated per feature.

Standard	Rule
One aggregate root per feature	incident (F1), investigation (F2–F3), root_cause (F4), resolution_plan (F5), incident_report (F6) — each is the single source of truth for its feature.
JSONB over premature normalization	AI-shaped outputs (planner_output, findings, supporting_evidence, steps) are stored as JSONB, not exploded into join-heavy tables, until proven necessary.
Structured Outputs everywhere	Every LLM call uses OpenAI Structured Outputs bound to a Pydantic model. Free-form "return JSON" prompting is disallowed.
One shared llm_execution table	Every AI call across all features logs to the same table: agent, model, prompt version, tokens, latency, cost, status.
One shared error contract	timestamp, status, error, code, message, path, correlationId — identical shape on every API in every feature.
investigationId-only internal calls	Spring Boot never forwards the full incident/investigation payload internally; the Python service always reloads from PostgreSQL by ID.
No agent-to-agent calls	All coordination between agents happens through LangGraph state, never direct calls.
Tool Adapter Layer	Agents never call Splunk/Prometheus/GitHub SDKs directly; every external system sits behind a common ToolAdapter interface.

8.2 Glossary

Term	Definition
Canonical Alert	The immutable, vendor-neutral alert model produced by Feature 1 from any monitoring tool payload.
Investigation Plan	The Planner's structured output (Feature 2): investigation type, priority, agents to run, evidence required, hypotheses.
Evidence	A single normalized fact returned by one specialist agent in Feature 3 (logs, metrics, or deployment finding).
InvestigationKnowledge	The deterministic output of the Evidence Intelligence Engine: a timeline, relationships, and health status derived from raw evidence, with no root-cause conclusion.
RootCauseAssessment	Feature 4's structured conclusion: the single most probable root cause, its confidence, supporting evidence, and alternatives.
ResolutionPlan	Feature 5's advisory remediation plan: strategy, risk, owning team, ordered steps, verification steps.
IncidentReport	Feature 6's human-readable post-incident document: executive summary, timeline, lessons learned, preventive actions.
LangGraph	The orchestration framework used to sequence and parallelize agent nodes and persist workflow state.
MCP	Model Context Protocol — a standardized tool-discovery protocol used as one option behind the Tool Adapter Layer.

8.3 Architecture Review Checklist

Recommended items for the architecture review meeting before implementation starts:

- Confirm the Exception Handling Standard, state machines, and backend sequence pattern (§2.7) as platform-wide, not Feature-1-only.
- Confirm the Evidence → Knowledge → Reasoning layering (§1.4, §4.4) as a hard boundary: no raw telemetry may reach an LLM prompt.
- Confirm Kafka vs. synchronous internal REST for the Feature 1→2 and Feature 2→3 handoffs (call-outs in §2.6, §3.6, §4.6).
- Confirm GPT-5.5 as the standing model choice and GPT-4.1 fallback policy for lightweight tasks (§1.5).
- Confirm the MVP/Enterprise boundary for Feature 6 (§7.4) — no vector DB or embedding pipeline in MVP scope.
- Confirm ownership of the Tool Adapter Layer (§4.7) and which observability integrations are needed for MVP launch.
- Confirm data retention and PII redaction policy for `raw_payload`, prompts, and `llm_execution` logs.

End of specification. Source: consolidated architecture design notes for AIR (Agentic Incident Response), covering Features 1–6 of the MVP. Recommended Confluence import: one parent page for Section 1, one child page per feature (Sections 2–7), and one child page for the Appendix.